

Sprawozdanie z projektu 4

Własne środowisko Gymnasium

Autorzy: Marcin Wolder, Stanisław Wojtas

11 maja 2026

1 Cel projektu

Celem projektu było przygotowanie własnego środowiska zgodnego z biblioteką Gymnasium, a następnie zbudowanie agenta, który potrafi skutecznie rozwiązywać zadanie występujące w tym środowisku. Jako temat wybrano uproszczoną wersję gry *Crossy Road*. Zadaniem agenta jest przeprowadzenie postaci przez kolejne pasy ruchu samochodowego bez kolizji i dotarcie do wyznaczonego celu.

Projekt spełnia wymagania zadania, ponieważ obejmuje:

- autorskie środowisko Gymnasium,
- implementację agenta uczącego się gry,
- eksperymenty porównujące wyuczoną politykę z zachowaniem losowym,
- tryb graficzny oparty na `pygame`.

2 Opis środowiska

Środowisko zostało zaimplementowane jako klasa `CrossyRoadEnv`. Plansza ma szerokość 10 pól i wysokość co najmniej 14 wierszy. Gracz rozpoczyna na bezpiecznym dolnym pasie, a jego celem jest osiągnięcie wiersza `goal_distance`. Pomiędzy startem a metą znajdują się pasy ruchu oraz losowo generowane bezpieczne odcinki.

Pełna przestrzeń akcji środowiska jest dyskretna i zawiera pięć akcji:

- ruch w górę,
- ruch w dół,
- ruch w lewo,
- ruch w prawo,
- oczekiwanie.

Samochody poruszają się po pasach z różnymi prędkościami i w obu kierunkach. Ich pozycje są aktualizowane w sposób ciągły, dlatego przestrzeń obserwacji nie została zbudowana jako mała siatka dyskretna, lecz jako wektor cech typu `Box`. W obserwacji znajdują się między innymi:

- znormalizowana pozycja gracza,
- informacja, czy ruch do przodu jest aktualnie ryzykowny,
- informacja, czy bieżąca pozycja za chwilę stanie się niebezpieczna,
- dla każdego pasa ruchu cechy najbliższego samochodu: względna odległość, prędkość i kierunek,
- lokalna mapa zajętości wokół gracza.

Tak zdefiniowana obserwacja pozwala agentowi podejmować decyzje na podstawie stanu ruchu drogowego, a nie tylko aktualnej pozycji na planszy. Jest to istotne, ponieważ poprawna gra wymaga przewidywania nadjeżdżających samochodów i czekania na bezpieczną lukę.

3 Funkcja nagrody

Funkcja nagrody została zaprojektowana tak, aby promować postęp oraz zachowania strategiczne. Agent otrzymuje:

- dodatnią nagrodę za przesuwanie się w stronę celu,
- dużą nagrodę za ukończenie poziomu,
- silną karę za kolizję,
- niewielką karę krokową za każdy ruch,
- dodatkowe kary za ruch wstecz, zbędne ruchy boczne i zbyt długie stanie w miejscu,
- premię za czekanie wtedy, gdy ruch do przodu byłby niebezpieczny,
- karę za wejście w ryzykowną sytuację mimo zbliżającego się zagrożenia.

Takie kształtowanie nagrody miało dwa cele. Po pierwsze, agent miał nauczyć się konsekwentnie zbliżać do mety. Po drugie, jego zachowanie miało wyglądać na celowe: wejście na jezdnię powinno następować wtedy, gdy jest to względnie bezpieczne, a nie losowo.

4 Algorytm uczący

Do rozwiązania problemu wykorzystano algorytm DQN z biblioteki Stable-Baselines3. Sieć była trenowana przez 100 000 kroków. Zastosowano między innymi następujące parametry:

- `learning_rate` = $3e-4$,
- `buffer_size` = 100000,
- `learning_starts` = 5000,
- `batch_size` = 64,

- `gamma = 0.99`,
- `train_freq = 4`,
- `target_update_interval = 500`.

W aktualnej wersji treningu użyto pełnej przestrzeni pięciu akcji, czyli `up`, `down`, `left`, `right` oraz `wait`. Dzięki temu agent może nie tylko przesuwać się naprzód, ale także wykonywać manewry korygujące pozycję i reagować na lokalny układ ruchu. Jest to ustawienie trudniejsze od wariantu z ograniczoną liczbą akcji, ponieważ powiększa przestrzeń decyzji i zwiększa ryzyko uczenia nieefektywnych zachowań bocznych.

Podczas treningu środowisko było okresowo ewaluowane, a najlepszy model zapisywano do pliku `best_model.zip`. Następnie właśnie ten najlepszy checkpoint był używany jako wynik końcowy.

5 Tryb graficzny

Środowisko obsługuje dwa tryby renderowania:

- `human` – interaktywne wyświetlanie gry w oknie,
- `rgb_array` – generowanie klatek obrazu do dalszego zapisu lub analizy.

Do renderowania wykorzystano bibliotekę `pygame`. Dzięki temu środowisko można uruchomić zarówno w trybie przeznaczonym dla człowieka, jak i w trybie przydatnym przy analizie działania agenta. Spełnia to wymaganie rozszerzenia projektu o warstwę graficzną.

6 Eksperymenty

Uczenie przeprowadzono dla limitu `max_steps = 500`. Według aktualnego podsumowania treningu agent rozegrał 959 epizodów, a średnia nagroda z ostatnich 20 epizodów wyniosła -0.5925. Najlepszy wynik ewaluacyjny uzyskany w trakcie treningu miał średnią nagrodę 8.439.

Końcowy model został porównany z bazą losową. Aktualna ewaluacja została wykonana dla 100 epizodów dla modelu oraz 100 epizodów dla polityki losowej. Wyniki zestawiono w tabeli 1.

Tabela 1: Porównanie wyuczonego agenta z bazą losową

Metryka	Agent DQN	Polityka losowa
Win rate	0.49	0.00
Collision rate	0.50	0.77
Truncation rate	0.01	0.23
Średnia nagroda	4.9868	—
Średnia liczba kroków / czas przeżycia	103.28	311.53
Udział akcji <code>up</code>	0.487	0.204
Udział akcji <code>wait</code>	0.341	0.199
Wait przy ryzykownym ruchu naprzód	0.579	0.201

Najważniejszy wniosek jest taki, że agent uczony metodą DQN nie działa losowo, ponieważ w 49% epizodów dociera do celu, podczas gdy polityka losowa nie wygrała ani razu. Dodatkowo współczynnik kolizji modelu jest wyraźnie niższy niż dla bazy losowej, a średnia nagroda modelu jest dodatnia. Oznacza to, że agent nauczył się częściowo rozwiązywać zadanie i robi to zauważalnie lepiej niż polityka przypadkowa.

Istotna jest także metryka *wait when forward risky rate*: model czeka w około 57.9% sytuacji, w których ruch do przodu był ryzykowny, natomiast polityka losowa robi to tylko w około 20.1% takich przypadków. Rozkład akcji także wskazuje na zachowanie nielosowe. Agent wyraźnie preferuje ruch w górę, ale często korzysta też z oczekiwania oraz sporadycznych ruchów bocznych i w dół. W porównaniu z polityką losową nie rozkłada akcji równomiernie, tylko wykazuje preferencje odpowiadające próbie osiągnięcia celu.

Warto jednak ostrożnie interpretować średni czas przeżycia. Polityka losowa ma tu wyższą wartość głównie dlatego, że w wielu epizodach nie kończy gry kolizją ani sukcesem, tylko dochodzi do limitu 500 kroków i zostaje ucięta. Z tego powodu dla tego zadania bardziej miarodajny jest *win rate* oraz analiza decyzji podejmowanych w sytuacjach ryzyka niż sama liczba kroków.

7 Wnioski końcowe

Projekt zakończył się powodzeniem. Udało się przygotować własne środowisko Gymnasium o bardziej złożonej strukturze niż prosta plansza dyskretna, wytrenować agenta oraz dodać tryb graficzny. Otrzymane wyniki pokazują, że agent nauczył się użytecznej, nielosowej polityki, potrafi ukończyć niemal połowę epizodów i osiąga wyraźnie lepsze wyniki niż baza losowa.

W przyszłości środowisko można rozwijać dalej, na przykład przez zwiększenie liczby typów przeszkód, dodanie dynamicznie zmieniających się pasów lub porównanie DQN z innymi algorytmami uczenia ze wzmocnieniem, takimi jak PPO. Już w obecnej postaci projekt spełnia jednak wymagania zadania i pokazuje pełny proces: od zaprojektowania środowiska, przez uczenie agenta, aż po ocenę jakości wyuczonej polityki.